

大模型知识扫盲

<https://zhuanlan.zhihu.com/p/1987568823277205130>

<https://zhuanlan.zhihu.com/p/1900918519597606471>

https://blog.csdn.net/a_yhr/article/details/147728299

放弃“智能体之年”的幻想，全面备战“**智能体十年**”的长期竞争。

AI的本质是模仿人类数据的“幽灵”，不是通过进化而来的“动物”。其能力边界由**数据**决定，而非生物本能。停止对模型“知识”的痴迷。

人类不擅长记忆是一种**特性** (Feature) 而非**缺陷** (Bug)。因为不善记忆，人类被迫去寻找更通用的模式和抽象规律。而LLM的超强记忆力反而使其分心，阻碍了更高层次的泛化。

——Andrej Karpathy

agent: <https://hello->

agents.datawhale.cc/#/./chapter11/%E7%AC%AC%E5%8D%81%E4%B8%80%E7%AB%A0%20Agentic-RL

<https://github.com/ThinPikachu/hello-agents>

提示词: <https://promtpilot.volcengine.com/>

一、模型参数与显存之间的关系

以Qwen2.5-7B模型（即Qwen 2.5系列中的70亿参数版本）为例，所需的显存（GPU memory）取决于多个因素，包括：

- 模型精度（FP16、INT4、INT8等）
- 是否使用推理优化技术（如vLLM、GGUF + llama.cpp、TensorRT-LLM、Hugging Face Transformers + accelerate等）
- 是否启用KV Cache（推理时缓存历史键值对）
- 批处理大小（batch size）和上下文长度（context length）

模型大小估算

模型参数数量为7B（70亿 = 7×10^9 ），不同精度下每个参数占用的字节数：

精度	每参数字节数	总模型大小（约）
FP32（32位浮点）	4 bytes	$7e9 \times 4 \approx 28 \text{ GB}$
FP16 / BF16	2 bytes	$7e9 \times 2 \approx 14 \text{ GB}$
INT8	1 byte	$\approx 7 \text{ GB}$
INT4	0.5 byte	$\approx 3.5 \text{ GB}$

因此单单加载一个FP16精度的Qwen2.5-7B模型就需要大约14GB的显存，再加上其他开销，那么总共的显存开销大约是16 ~ 18GB。

一个经验公式（仅推理，batch=1）：

显存 ≈ 模型权重大小 + (2 × 参数量 × 序列长度 × 精度字节数) / 1e9（用于 KV Cache 估算）

为什么“模型权重”占用显存？

当你加载一个模型（如 Qwen2.5-7B）到 GPU 时：

- 1. 权重必须从硬盘加载到显存中，才能进行计算。
- 2. 每个权重是一个数字（如 float16 占 2 字节），70 亿个 × 2 字节 ≈14 GB。
- 3. 推理时还需要额外空间存储中间结果（激活值、KV Cache 等），所以总显存 > 权重大小。

二、部署方式

vLLM

SGLang

三、大模型微调

3.1 预训练（Pre-training）与微调（Fine-tuning）区别

- 预训练：模型初始阶段的大规模通用训练，学习广泛的知识。
- 微调：针对具体任务，利用小型的特定数据集进行二次训练。
- 严谨定义：训练大模型的两阶段范式。预训练是在海量通用数据上进行无监督学习以获得通用知识；微调则是在规模更小、有特定标签的数据上进行监督学习以适应特定任务。
- 通俗解释：预训练就像“通识教育”，让AI把互联网上的书都读一遍，学会说话和思考。微调就像“专业深造”，用特定领域的资料（比如公司内部文档）把它训练成某个行业的专家。

3.2 全参数微调 (Full Fine-tuning)

这是一种最朴素也最暴力的方式——解冻预训练模型的所有参数，在下游数据上继续训练，使模型完整学习新任务。它的优点是在足够数据下能够获得最充分的适应效果，模型可以自由调整每一层参数来拟合新任务；但缺点也显而易见：资源消耗巨大（需要显存随模型大小线性增加，14B以上模型往往单卡无法全参数微调），过拟合风险更高（尤其当下游数据较少时，大量参数容易记忆训练集导致泛化变差）。另外，全量微调后的模型参数完全改变，如果要服务多个任务需要保存多份完整模型，部署成本高。一般来说，全参数微调适用于小模型或下游数据非常丰富且有充足计算资源的情况。在大模型上，由于效率太低，我们更推荐LoRA/QLoRA这类参数高效微调手段。

- **原理：**更新所有模型参数。
- **优点：**性能最好，保留原始能力的同时适配新任务。
- **缺点：**显存和算力消耗极大
- **适用场景：**资源充足、对性能要求极高的场景（如企业级私有模型）。

3.3 高效微调 (Parameter-Efficient Fine-Tuning, PEFT)

方法	原理	可训练参数占比	特点
LoRA (Low-Rank Adaptation)	在权重矩阵旁路添加低秩分解矩阵 ($A \times B$)	~0.1%–1%	简单有效，推理无延迟，支持合并权重
QLoRA	LoRA + 4-bit 量化 + 分页优化器	~0.1%	可在 24GB 显存跑 65B 模型微调
Adapter	在 Transformer 层中插入小型神经网络模块	~1%–3%	推理有轻微延迟，需修改模型结构
Prefix Tuning / Prompt Tuning	在输入前添加可学习的“软提示”向量	<0.1%	无需改模型，但效果略弱于 LoRA

LoRA (Low-Rank Adaptation, 低秩适配) (主流方案)

LoRA通过在模型的部分权重上添加可训练的低秩矩阵来实现微调。简单来说，就是冻结原模型的大部分参数，仅在每层中引入很小的瓶颈层进行训练。这样大大减少了需要更新的参数数量。LoRA的优点是内存开销小、训练高效，在下游任务上的效果通常接近全参数微调。并且多个LoRA适配器可以在一个基模型上切换，方便一个模型服务多种任务。LoRA不增加推理时延，因为微调完可以将低秩权重与原权重合并。实验表明，使用LoRA微调后的性能往往与全量微调相当，但显存占用和计算量却显著降低。缺点是LoRA仍需加载完整的预训练模型作为基础（但可以使用8-bit/4-bit量化减小内存，占用稍高于更轻量的P-Tuning）。LoRA适用于中大型模型在中小规模数据上的高效微调，是目前社区中极为流行的方案。

参数高效微调 PEFT

LoRA: Low-Rank Adaptation

给模型做"微创手术"而非"全身手术"

✗ 传统微调：全身手术

层1: 10亿参数 📦

层2: 10亿参数 📦

层3: 10亿参数 📦

... 共175B参数全部训练

💰 成本: \$10,000+

🕒 时间: 数天

VS

☑ LoRA：微创手术

🔒 原始模型（冻结）

层1: 10B ❄️

层2: 10B ❄️

层3: 10B ❄️

不改变参数

+

💊 LoRA补丁（训练）

A

×

B

仅0.1%参数!

♥️ LoRA优势

💰 成本: \$10-100

🕒 时间: 几小时

💾 存储: 仅几MB

🔗 LoRA的巧妙之处

1. 原始权重矩阵很大 (如 1024×1024)
2. LoRA用两个小矩阵相乘来近似: $1024 \times 8 \times 8 \times 1024$
3. 参数量从100万降到1.6万 (减少98%!)
4. 推理时可以合并, 不增加延迟

🚀 典型应用

🌐 风格适配

不同写作风格

🏢 企业定制

专属知识库

🌐 多语言

特定语言优化

💡 用1%的成本，达到90%的效果

QLoRA (Quantized LoRA, 量化 LoRA)

QLoRA可以看作是在LoRA基础上的进一步优化。它的核心是在训练时将预训练模型权重以4比特精度加载，大幅降低显存占用，同时同样仅训练LoRA低秩适配器。创新之处在于4-bit量化采用了Norm浮点格式(NF4)等技术，尽可能减少量化带来的性能损。研究表明，QLoRA在单张48GB GPU上就能微调65B参数的模型，而且性能与全16位精度微调几乎持平！这意味着，即使只有一块高端GPU，也能微调过去需要数十张GPU的大模型，这对普通开发者来说是革命性的。QLoRA的优点是极致地节省显存（比LoRA进一步减半左右），使单卡可微调更大的模型。缺点是实现稍复杂，需要依赖如 `bitsandbytes`、可能还需要DeepSpeed等库支持4-bit训练，而且由于进行了强烈的量化，极少数情况下可能出现略微的性能下降或兼容性问题。QLoRA非常适合GPU内存非常有限但又想微调超大模型的情况。例如只有一块16GB卡却希望微调13B或33B参数模型时，QLoRA是不错的选择。

指令微调 (Instruction Tuning) (Prompt Tuning)

P-Tuning是一种Prompt学习方法，通过为模型的输入添加一些可训练的虚拟token来引导模型输出。与直接调整模型权重不同，P-Tuning让模型保持原有权重不变，只是在输入序列开头插入若干新参数（这些参数在微调时会被更新）。可以理解为我们为每个任务学到了一个“魔法开头咒语”，让预训练模型更好地完成特定任务。P-Tuning的参数规模非常小（只相当于几百个词的嵌入向量），因此训练开销极低。它的优点是实现简单、所需内存极小，非常适合极少样本（Few-Shot）或需要针对很多不同提示调优的场景。缺点是适用范围受限：由于只调整输入提示，模型本身的表示能力没有改变，因而对于复杂任务或需要模型深度调整的场景，效果不如LoRA或全量微调。此外，P-Tuning主要针对生成类任务（prefix-tuning用于NLG，P-Tuning最初用于NLU任务）。总的来说，P-Tuning适合小数据快速尝试，或者配合其他微调一起使用，以进一步提升性能。

四、RAG是什么

https://www.zhihu.com/tardis/zm/art/675509396?source_id=1003

一句话总结：

RAG (中文为检索增强生成) = 检索技术 + LLM 提示。

每当将大模型应用于实际业务场景时发现，通用的基础大模型基本无法满足实际业务需求，主要有以下几方面原因：

- 知识的局限性：大模型自身的知识完全源于训练数据，而现有的主流大模型（deepseek、文心一言、通义千问…）的训练集基本都是构建于网络公开的数据，对于一些实时性的、非公开的或私域的数据是没有。

- 幻觉问题：所有的深度学习模型的底层原理都是基于数学概率，模型输出实质上是一系列数值运算，大模型也不例外，所以它经常会一本正经地胡说八道，尤其是在大模型自身不具备某一方面的知识或不擅长的任务场景。
- 数据安全性：对于企业来说，数据安全至关重要，没有企业愿意承担数据泄露的风险，尤其是大公司，没有人将私域数据上传第三方平台进行训练会推理。这也导致完全依赖通用大模型自身能力的应用方案不得不在数据安全和效果方面进行取舍。

而RAG（Retrieval-Augmented Generation，检索增强生成）是一种结合 信息检索（Retrieval）与 大语言模型生成（Generation）的人工智能技术框架，旨在让模型在回答问题时能够基于外部知识库中的真实、最新信息进行生成。

RAG 是 **Retrieval-Augmented Generation**（检索增强生成）的缩写。

用大白话讲，RAG 就是给大模型（LLM）配了一个“外挂知识库”或“开卷考试的小抄”。

1. RAG 是什么？（核心概念）

大模型（如 GPT-4、Claude）虽然博学，但它有两个致命弱点：

知识截止：它只训练到某个时间点之前的数据，不知道今天的新闻或你公司的内部文档。

幻觉：当它不知道答案时，喜欢瞎编。

RAG 的作用：在你问问题之前，系统先去你的私有数据库里把相关的资料找出来，然后把这些资料和问题一起塞给大模型，说：“根据这些资料，回答这个问题。”

类比：

普通 LLM：像一个闭卷考试的学生，全靠脑子记，容易记错或编造。

RAG：像一个允许带参考书进考场的学生。他先翻书（检索），找到相关段落，然后结合自己的理解写出答案（生成）。

2. RAG 的工作流程（怎么用？）

实现一个 RAG 系统通常分为两个阶段：**数据处理阶段（离线）** 和 **问答阶段（在线）**。

● 阶段一：数据处理（建立索引）

这是在使用前一次性做好的准备工作。

加载（Load）：读取你的数据源（PDF、Word、网页、数据库、代码文件等）。

切片（Split/Chunk）：因为上下文窗口有限，不能把整本书塞进去。要把长文档切成小块（Chunk），比如每 500 个字一块，块与块之间重叠一点以保持连贯性。

向量化（Embedding）：

使用一个专门的“嵌入模型”（Embedding Model，如 `text-embedding-ada-002` 或开源的 `bge-m3`），把每一块文本转换成一组数字（向量）。

原理：语义相似的句子，转换后的向量在空间距离上会很近。

存储（Store）：把这些向量存入向量数据库（Vector DB，如 Chroma, Pinecone, Milvus, FAISS, Qdrant）。

● 阶段二：问答流程（实时检索）

这是用户每次提问时发生的步骤。

用户提问：用户输入：“我们的报销政策是什么？”

查询向量化：把这个问题也转换成向量。

检索（Retrieve）：

在向量数据库中搜索，找出与“问题向量”最相似的 Top-K 个文本块（比如最相关的 3 段政策原文）。

组装 Prompt（Augment）：

构造一个新的 Prompt，结构如下：

[系统指令] 你是一个助手。请仅根据以下提供的背景信息回答问题。如果信息不足，请说不知道。

[背景信息]

- [背景信息]
1. 员工差旅报销需在行程结束后7天内提交...
 2. 餐饮报销上限为每天200元...
 3. 发票必须为增值税专用发票...

[用户问题]
我们的报销政策是什么？

生成 (Generate) :

把这个组装好的 Prompt 发给大模型 (LLM) 。

LLM 基于背景信息生成最终答案。

3. 如何用代码实现一个简单的 RAG?

目前最流行的框架是 **LangChain** 或 **LlamaIndex**。下面是一个使用 LangChain + ChromaDB 的最小化 Python 示例：

第一步：安装依赖

```
pip install langchain langchain-community chromadb sentence-transformers openai
```

第二步：编写代码

```
import os
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import CharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings # 或使用 OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI # 假设使用 OpenAI

# 1. 准备数据 (假设有一个 policy.txt 文件)
loader = TextLoader("policy.txt")
documents = loader.load()

# 2. 切片 (Split)
text_splitter = CharacterTextSplitter(chunk_size=500, chunk_overlap=50)
docs = text_splitter.split_documents(documents)

# 3. 向量化并存储 (Embed & Store)
# 这里使用本地开源嵌入模型，也可以换成 OpenAIEmbeddings
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
vectorstore = Chroma.from_documents(docs, embeddings)

# 4. 构建 RAG 链 (Retrieve & Generate)
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0) # 温度设为0以保证严谨

# RetrievalQA 是 LangChain 封装好的标准 RAG 链
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff", # "stuff" 表示把所有检索到的片段直接塞进 Prompt
    retriever=vectorstore.as_retriever(search_kwargs={"k": 3}) # 取最相关的3个片段
)

# 5. 执行查询
query = "出差报销的截止时间是多少？"
result = qa_chain.invoke({"query": query})

print(result['result'])
```

4. RAG 的关键优化点 (进阶)

简单的 RAG 往往效果不好，需要优化：

问题	优化手段	说明
检索不准	混合检索 (Hybrid Search)	结合“关键词匹配” (BM25) 和“向量相似度”，提高召回率。
切片过大 (太大)	语义切片 (Semantic Chunking)	不按固定长度切，而是按段落、标题或语义完整

切片太碎/太大	语义切片 (Semantic Chunking)	不按字符数切，而是按段落、标题或语义完整性切分。
丢失上下文	父文档检索 (Parent Document Retriever)	检索时用小块定位，但返回给 LLM 的是包含该小块的更大范围文本（父文档）。
多跳推理难	Query Rewrite / HyDE	如果用户问得模糊，先用 LLM 改写问题，或者先生成一个假设性答案再去检索。
来源不可信	引用溯源	让 LLM 在回答中标注 [Source: doc1.pdf, page 5]，方便用户核对。

5. RAG 与你之前问的“结构化笔记”有什么区别？

这是一个非常好的对比！

特性	RAG (检索增强生成)	结构化笔记 (Structured Notes)
数据类型	非结构化为主 (PDF, 文章, 日志, 代码全文)	结构化为主 (Key-Value, JSON, TODO列表, 决策记录)
检索方式	语义相似度 (Vector Search) *"找意思相近的"*	精确匹配/关键词 (Key Lookup / SQL) *"找指定ID的"*
适用场景	知识库问答 (Q&A) "帮我总结这份合同的风险"	状态管理/记忆 (Memory) "上次决定的DB密码是什么?"
准确性要求	允许一定的模糊性，重在归纳总结	要求绝对准确，不能有任何偏差
更新频率	低频更新 (文档很少变)	高频更新 (随任务进展随时写入)

最佳实践： 在一个复杂的 Agent 系统中，两者通常共存：

- 用 **RAG** 来查阅公司的手册、技术文档、历史代码库。
- 用 **结构化笔记** 来记录当前任务的进度、临时决策、变量状态。

总结

RAG 是让大模型“读懂”你私有数据的最佳方案。如果你有一堆 PDF、Wiki 或代码仓库想让 AI 理解并问答，就用 RAG。如果你想让 AI 记住它自己做过什么、决定过什么，就用结构化笔记。

五、模型量化

一种模型压缩技术，通过降低模型权重和激活值的数值精度（如从32位浮点数降至8位或4位整数）来减小模型体积和加速推理。通俗解释就是像把一张高清无损图片（原模型）压缩成JPG格式（量化后的模型）。文件变小了很多，加载更快，虽然画质有极细微损失，但肉眼几乎看不出来。这让大模型能在手机上运行。

模型量化

Quantization

像压缩图片一样压缩AI模型

🖼️ 类比：图片压缩

高清无损图
100MB RAW格式
极高画质

压缩

JPG图片
2MB文件
肉眼看不出差别

🔍 量化原理：降低数值精度

原始：32位浮点数 (FP32)

3.141592653589793238462643383279...
每个权重占32位，极高精度，文件巨大

量化：8位整数 (INT8)

3.14
每个权重占8位，精度够用，文件缩小75%!

📊 常见量化级别

FP32

32位
100% 大小
最高精度

FP16

16位
50% 大小
微小损失

INT8

8位
25% 大小
几乎无损

INT4

4位
12.5% 大小
轻微损失

🔗 实际效果

例：Llama 70B模型

FP32: 280GB ❌ 服务器才能跑 INT4: 35GB ✅ 消费级显卡

推理速度：提升2-4倍 | 准确率：保持98%+ | 内存占用：减少75%

🎯 应用场景

📱 手机部署

让大模型在
手机上运行

🏠 边缘计算

在本地设备
离线运行

💰 降低成本

减少服务器
GPU需求

💡 核心价值

牺牲极小精度，换取巨大的效率提升

六、知识蒸馏

知识蒸馏（Knowledge Distillation, KD）是一种模型压缩与迁移学习范式，其核心思想是通过将一个高容量、高性能的教师模型（Teacher Model）所编码的暗知识（dark knowledge）——即其输出概率分布中蕴含的类别间相似性与不确定性信息——迁移至一个结构更紧凑、计算效率更高的学生模型（Student Model），从而在显著降低模型复杂度与推理成本的同时，尽可能保留原始模型的泛化能力与预测性能。

知识蒸馏

Knowledge Distillation

宗师传功给徒弟的AI版本

🏯 武侠小说的传功场景

老宗师（大模型）把毕生功力传给年轻徒弟（小模型）

徒弟体格虽小，但继承了宗师的精髓

📦 蒸馏过程

👤 教师模型

GPT-4级别

- 175B参数
- 极高准确率
- 运行成本高
- 需要大服务器
- 响应慢

传授知识

- ⚡ 输出概率
- ⚡ 中间特征
- ⚡ 决策逻辑

🧒 学生模型

轻量级版本

- 7B参数 (小95%)
- 保持90%性能
- 成本降低10倍
- 可在手机运行
- 响应快速

🔑 蒸馏的秘密

- 1 **软标签学习**
不只学"对错", 还学"可能性分布"
- 2 **中间层对齐**
模仿老师的"思考过程", 不只是答案
- 3 **温度参数**
让知识更"柔和", 便于学生吸收

📊 真实效果对比

模型	参数量	准确率
BERT-Large	340M	95%
DistilBERT	66M (-80%)	92% (-3%)

⚡ 应用场景

📱 端侧部署

手机/IoT
设备运行

⚡ 实时服务

低延迟
快速响应

💰 降成本

减少算力
规模化部署

七、减枝

剪枝（Pruning）是一种模型压缩技术，其核心思想是移除神经网络中“不重要”或“冗余”的权重、神经元、通道或层，从而减小模型规模、降低计算量和内存占用，同时尽量保持模型性能。

神经网络通常存在高度冗余性：

- 某些权重接近 0，对输出影响微乎其微；
- 某些神经元几乎不激活；
- 某些注意力头（attention head）贡献极小

通过识别并移除这些部分，可在精度损失很小的情况下显著压缩模型。

八、swift

ms-swift是魔搭社区提供的大模型与多模态大模型微调部署**框架**，现已支持600+纯文本大模型与300+多模态大模型的训练（预训练、微调、人类对齐）、推理、评测、量化与部署。其中大模型包括：Qwen3、Qwen3-Next、InternLM3、GLM4.5、Mistral、DeepSeek-R1、Llama4等模型，多模态大模型包括：Qwen3-VL、Qwen3-Omni、Llava、InternVL3.5、MiniCPM-V-4、Ovis2.5、GLM4.5-V、DeepSeek-VL2等模型。

<https://swift.readthedocs.io/zh-cn/latest/GetStarted/Quick-start.html>

九、准确率，召回率，精确率

场景：你是个“找猫志愿者”，任务是从一堆动物里找出所有 **真正的猫**。

- 实际情况：院子里有 **10 只真猫 + 90 只狗**（共 100 只动物）。
- 你努力找了一圈，指着 **15 只动物**说：“这些是猫！”
- 但其实：
 - 你指的 15 只里，**只有 7 只是真猫**，
 - 另外 **8 只是狗**（你认错了），
 - 而剩下的 **3 只真猫**，你没发现，漏掉了。

现在来看三个指标：

1. **精确率**（Precision）→ “你抓的里面，有多少是真的？”

- 你抓了 15 只，其中 7 只是真猫
→ **精确率** = $7 \div 15 \approx 47\%$

关注：**别乱抓！**

精确率低 = 把很多狗当猫（误报多）

2. 召回率 (Recall) → “所有真猫，你找到了几个？”

- 总共有 10 只真猫，你只找到 7 只
→ **召回率** = $7 \div 10 = 70\%$

关注：**别漏掉！**

召回率低 = 漏掉真猫（漏报多）

3. 准确率 (Accuracy) → “你总共判断对了多少？”

- 你判断了 100 只动物：
 - 正确的有：7 只真猫（抓对） + 82 只狗（没抓，也算对） = 89 只
 - 错误的有：8 只狗被抓 + 3 只猫漏掉 = 11 只
- **准确率** = $89 \div 100 = 89\%$

一句话总结三者区别：

名称	问的问题	关心什么
精确率 (Precision)	“你抓的里面，有几个是真的？”	别抓错（少冤枉）
召回率 (Recall)	“所有真的，你抓到了几个？”	别漏掉（全覆盖）
准确率 (Accuracy)	“你总共答对了几题？”	整体正确率

-
- 精确率** = “抓得准不准”
 - 召回率** = “找得全不全”
 - 准确率** = “整体对不对”

十、微调也就是ft或者sft属于监督学习

方法	数据要求	示例
微调	需要 输入-输出对 $((x, y))$, 即“问题-标准答案”	- 用户提问: “如何煮咖啡?” → 标准回答: “先烧水, 再…” - 中文 → 英文翻译对
强化学习	只需要 输入 + 奖励信号 (x, r) , 不需要标准答案	- 用户提问: “如何煮咖啡?” → 模型生成回答 → 人类打分 (👍/👎) 或自动评分 (如流畅度 + 相关性) → 奖励值 r

维度	微调	强化学习 (如 PPO + RLHF)
损失函数	交叉熵 (Cross-Entropy)	策略梯度 (Policy Gradient) , 如 PPO
训练稳定性	高 (梯度平滑)	低 (奖励稀疏、方差大)
计算成本	较低	极高 (需多次生成 + 人工/模型打分)
是否需要人类反馈	否 (只需标注数据)	是 (RLHF 需要人类偏好数据)
代表技术	SFT (Supervised Fine-Tuning)	RLHF (Reinforcement Learning from Human Feedback)

十一、机器学习与深度学习

1. 核心定义与关系

- 人工智能 (AI): 大概念, 指任何让机器展现智慧的技术。
- 机器学习 (ML): AI 的一个子集。它不是让机器“死记硬背”规则, 而是让机器通过算法从大量数据中寻找规律, 从而对未知数据进行预测或决策。
- 深度学习 (DL): ML 的一个子集。它特指利用多层人工神经网络 (Artificial Neural Networks) 来模拟人脑学习过程的技术。

2. 关键区别: 通过一个例子来理解

假设我们要让电脑识别一张照片里有没有猫。

- 传统机器学习的做法:
 - 人工特征提取 (关键步骤): 你 (工程师) 需要先告诉电脑: “猫有尖耳朵”、“猫有胡须”、“猫的眼睛是圆的”。你需要手动编写代码来提取这些特征。
 - 分类: 电脑根据这些你定义好的特征, 判断这是不是猫。

- **局限性**：如果猫侧着脸，或者光线很暗，导致你定义的特征提取失败，识别率就会大幅下降。
- **深度学习的做法**：
 - **端到端学习 (End-to-End)**：你不需要告诉电脑“猫有耳朵”。你只需要把成千上万张猫的照片丢给一个深层的神经网络。
 - **自动特征提取**：神经网络的第一层可能识别到了“线条”，第二层识别到了“形状”，第三层识别到了“耳朵”和“眼睛”的组合……它自己学会了什么是猫。
 - **优势**：数据量越大，效果越好，能处理极度复杂的非结构化数据（图像、文本、声音）。

3. 总结对比表

维度	机器学习 (Traditional ML)	深度学习 (Deep Learning)
数据量需求	较小数据量也能工作	需要海量数据才能发挥威力
算力需求	CPU 通常即可	几乎必须使用 GPU 进行并行计算
核心算法	决策树、SVM、逻辑回归、随机森林	卷积神经网络(CNN)、Transformer、RNN
特征工程	主要瓶颈，需要人工提取特征	自动提取，由网络自己学习
可解释性	较强，知道为什么判别	较弱，“黑盒”模型，难以解释中间过程

十二、目标函数和损失函数

损失函数 (Loss Function)，它是衡量模型在单个样本上预测结果与真实结果之间差距的指标。想象你在练习射箭，靶心是真实值（Ground Truth），你的箭射中的位置是预测值（Prediction），射偏了多少米？这个距离就是“损失”。如果你射中了靶心，损失就是 0。如果你射到了靶子外面，损失就很大。

目标函数 (Objective Function)，它是最广义的概念，指我们在优化过程中最终想要最小化或最大化的那个函数，这是你的终极考核指标。通常情况下，你的目标就是让“代价”越小越好（让箭射得越准越好）。但是，有时候为了防止模型“死记硬背”（过拟合），我们会加一些约束条件（正则化项）

- 比如：目标 = (射箭的平均误差) + (姿势的标准化程度)。
- 这里，“让姿势标准”就是为了保证你换把弓也能射准，而不仅仅是用这把特定的弓准。

1. 什么是“最小化”和“最大化”？

这其实就是我们做任何事情的两种心态。

场景一：你想省钱（最小化）

想象你在淘宝买东西。

- 你的目标是什么？花最少的钱。
- 这时候，你的目标函数就是： $\text{总价格} = \text{商品价格} + \text{运费}$ 。
- 你在不停地换商家、找优惠券，目的是什么？是为了让这个“总价格”变得越小越好。
- 这就是最小化 (Minimize) 。

场景二：你想考高分（最大化）

想象你在打王者荣耀或者 LOL。

- 你的目标是什么？赢，或者杀更多的人。
- 这时候，你的目标函数就是： KDA (击杀/死亡/助攻比) 。
- 你在游戏里各种操作、走位，目的是什么？是为了让这个“KDA”数值越大越好。
- 这就是最大化 (Maximize) 。

2. 在 AI 里，这俩是一回事

在 AI 训练里，虽然叫法不同，但本质都是为了让模型“表现更好”。

- 最大化准确率 (Maximize Accuracy): 我希望模型考 100 分。
- 最小化错误率 (Minimize Error): 我希望模型别犯错（犯错次数为 0）。

这其实是硬币的两面：

如果你把“错误”降到了 0（最小化），你的“准确率”自然就是 100%（最大化）。

为什么 AI 里通常只说“最小化损失”？

因为“找茬”比“夸奖”更容易量化。让计算机去算“你这就差了 0.01 毫米就射中靶心了”，比算“你射得有多准”要方便数学计算。

现在的 AI 大模型（如 ChatGPT），本质上是在玩一个“猜字游戏”，它的目标函数就是“猜错下一个字的概率”，它一直在努力把这个概率“最小化”。

一句话总结：损失函数是“报错器”，告诉模型错得有多离谱；目标函数是“指挥棒”，指引模型往哪个方向修改参数，才能让整体错误最少。

十三、回归和分类

在数学和 AI 中，回归是指确定两个或多个变量之间定量关系的方法。

- 通俗理解：回归就是寻找一种“规律”，让你能根据已知的信息，推算出一个连续的数值。
- 生活例子：* 如果你根据“地段、面积、楼层”来预测“房价（比如 500 万）”，这就是回归。

回归问题 vs 分类问题

这是机器学习中最基础的两大任务，区别在于你输出的结果是什么：

1. 分类问题 (Classification): 输出的是类别（标签）。
 - 结果是离散的：是/否，猫/狗/猪，及格/不及格。
 - 就像做单选题。

2. 回归问题 (Regression): 输出的是具体的数。

- 结果是连续的: 1.5, 100.2, -5.0。
- 就像做填空题 (填数字)。

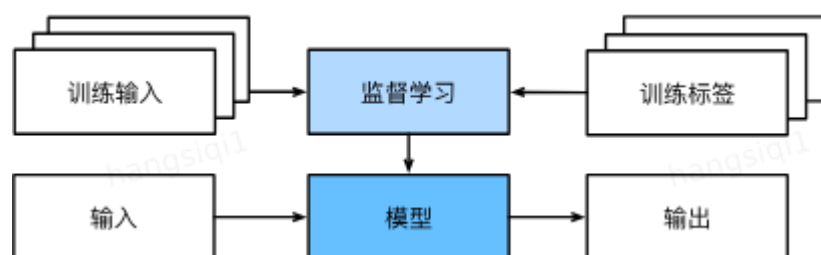
常见的回归应用场景

- 金融: 预测股票明天的价格。
- 气象: 预测下雨的具体降水量 (毫米)。
- 物流: 预测快递到达需要的具体分钟数。

十四、监督学习与无监督学习

它们本质上的区别在于: 训练数据里有没有“标准答案” (标签/Label)。

监督学习 (Supervised Learning)



定义: 我们给 AI 一堆数据, 并且明确告诉它每一个数据对应的“正确答案”是什么。

- 形象比喻: 就像学生做有参考答案的练习题。学生先看题目, 给出答案, 再对一下参考答案, 发现错了就去修改自己的知识点 (调整权重)。
- 核心逻辑:
 - 输入: x (题目)
 - 标签: y (答案)
 - 目标: 寻找一个函数 $f(x)$, 使得预测出来的 y 尽可能接近真实的 y 。

常见的监督学习任务:

1. 分类 (Classification): 预测类别。比如: 预测这张图片是猫还是狗; 预测这封邮件是不是垃圾邮件。
2. 回归 (Regression): 预测连续数值。比如: 预测明年的房价; 预测明天的气温。

无监督学习 (Unsupervised Learning)

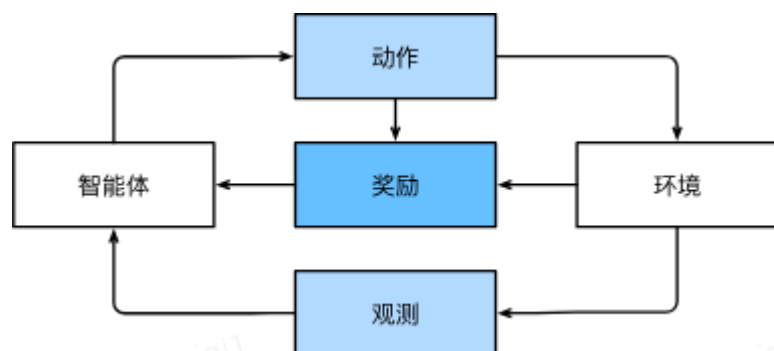
定义: 我们只给 AI 数据, 但不告诉它这些数据是什么, 也不给它标准答案。让 AI 自己去发现数据内部隐藏的结构或规律。

- 形象比喻: 就像把一个婴儿放进一堆玩具里。没人告诉他哪个是“积木”, 哪个是“皮球”, 但婴儿玩久了会发现: 这一堆东西是硬的、方的; 那一堆东西是软的、圆的。他自己把东西归类了。
- 核心逻辑:
 - 输入: 只有 x , 没有 y 。
 - 目标: 寻找数据之间的相似性、聚类或降维。

常见的无监督学习任务：

3. 聚类 (Clustering): 把相似的东西分到一组。比如：银行根据消费习惯把客户分成“高消费人群”和“省钱达人”，但并不提前定义这些人群的具体特征。
4. 降维 (Dimensionality Reduction): 把复杂的信息压缩。比如：把 100 个特征压缩成 3 个核心特征，方便可视化。
5. 关联分析：发现“买尿布的人通常也会买啤酒”。

十五、强化学习



十六、cnn rnn transformer diffusion

这四个架构是深度学习发展史上的四座里程碑，它们分别统治了图像、序列、语言和生成四大领域。

按时间线和功能，我们可以这样拆解：

1. CNN (Convolutional Neural Networks) —— 图像之王

核心思想：局部感知与参数共享。

CNN 模拟了人类视觉系统的原理。它通过“卷积核”（一个小窗口）在图片上滑动，去捕捉局部的特征（如线条、边缘、纹理）。

- 擅长领域：计算机视觉 (CV)。比如你的 YOLO_detect 项目，底层就是基于 CNN。
- 特点：具有“平移不变性”（不管猫在图片的左上角还是右下角，CNN 都能识别出来）。
- 形象比喻：像是一个拿着放大镜的侦探，一寸一寸扫视照片，寻找线索。

2. RNN (Recurrent Neural Networks) —— 序列老兵

核心思想：循环记忆。

RNN 是为了处理“序列数据”（有先后顺序的数据）而生的。它在处理当前信息时，会把之前的信息也作为输入传递下去。

- 擅长领域：语音识别、文本预测（早期）。
- 致命弱点：“记性不好”（梯度消失/爆炸）。如果你给它一段很长的文字，它读到结尾时已经把开头忘光了。
- 形象比喻：像是一个一边读一边忘的读者，虽然他在努力联系上下文，但书太厚他就晕了。

3. Transformer —— 现代 AI 的灵魂

核心思想：自注意力机制 (Self-Attention)。

它是目前大模型（GPT, Llama）的基础。它彻底取代了 RNN。它不再“按顺序阅读”，而是**“一眼看全局”**。

- 核心优势：
 - 并行计算：可以同时处理整段文字，速度极快。
 - 长距离依赖：它能精准捕捉到文章开头的一个“他”指代的是文章结尾的哪个“名字”。
- 擅长领域：大语言模型（LLM）、多模态。
- 形象比喻：像是一个拥有上帝视角的考官，一眼扫过去，瞬间划出整篇文章里所有相关的重点，并建立联系。

4. Diffusion (Diffusion Models) —— 艺术生成家

核心思想：去噪还原。

它是 Stable Diffusion 和 Midjourney 等绘图 AI 的核心算法。它的学习过程非常奇特：

- 加噪：把一张清晰的画不断加噪音，直到变成一团乱码。
 - 去噪（模型学的）：让 AI 学习如何把这团乱码一步步还原成清晰的画。
- 擅长领域：图像生成、视频生成。
 - 特点：生成能力极强，细节丰富。
 - 形象比喻：像是一个雕塑家，从一块杂乱无章的石头（噪声）中，一点点剔除多余部分，最后变出一尊精美的塑像。

总结对比

架构	核心能力	代表应用
CNN	空间特征提取	YOLO
RNN	简单序列记忆	早期机器翻译, 股票预测
Transformer	全局关联理解	ChatGPT, Claude, Llama
Diffusion	高质量内容生成	Stable Diffusion, Dall-E